

GRoS - Geomechanical Reservoir Simulator

Theoretical guide

GRoS developers

May 22, 2026

1 GRoS developers

- Stefano Nardean — former Postdoctoral researcher at UniPD, now at ENI S.p.A.
- Daniele Moretto — PhD candidate at UniPD
- Eduardo S. Castro — Postdoctoral researcher at UniPD

2 Introduction

This guide is intended to give a brief overview on the models implemented in GRoS. The presentation is not rigorous but mainly focused on implementation. Hence, we will omit details on functional setting of the equations. When necessary, we will make reference to a piece of GRoS code.

In section 3 we start with a general overview of the discretization techniques implemented in GRoS, while the specific formulation of the available Physics models is described in section 4.

Finally, section 5 explains the core concepts of interface models and the related implementation details.

3 Discretization

We start with a general overview of the discretization techniques implemented in GRoS, namely the Finite Volume method (with Two-Point Flux Approximation, FV-TPFA) and Lagrangian Finite Elements (FEM). Consider the classical Poisson equation

$$\nabla \cdot \nabla u = g \quad \text{in } \Omega \quad (1)$$

We will show how both FVTPFA and FEM allow us to move from the continuous equation (1) to the solution of the resulting linear system

$$A\mathbf{u} = \mathbf{g} \quad (2)$$

We will also discuss the main ingredients of the corresponding GRoS implementation. Both methods work on an integral form of the equation

$$\int_{\Omega} \nabla \cdot \nabla u \, d\Omega = \int_{\Omega} g \, d\Omega \quad (3)$$

We will introduce a discretization $\mathcal{K}$ of Ω , for which:

- $k \in \mathcal{K}$ denotes each individual cell of the discretization.
- $\mathcal{N}$ is the set of nodes/vertices of $\mathcal{K}$, indexed as $i \in \mathcal{N}$.
- $\mathcal{F}$ is the set of faces of $\mathcal{K}$, indexed as $f \in \mathcal{F}$.
- $\mathcal{E}$ is the set of edges of $\mathcal{K}$, indexed as $e \in \mathcal{E}$.

In the following, with an abuse of notation, we will use the same symbol to indicate geometrical element itself and the corresponding set of indices/dofs in matrices. For the previously defined set, we use the following notation for incidence relationship between elements of different sets:

$$\mathcal{A}^b : \{a \in \mathcal{A} : a \text{ is incident to } b\}, b \in \mathcal{B}$$

For example

- $\mathcal{F}^i$ is the subset of faces in $\mathcal{F}$ sharing node i
- $\mathcal{E}^i$ is the subset of edges in $\mathcal{E}$ sharing node i
- $\mathcal{F}^k$ is the subset of faces in $\mathcal{F}$ bounding cell k .
- $\mathcal{N}^f$ is the subset of nodes in $\mathcal{N}$ being vertices of face f .

and so on. Let $\Gamma = \partial\Omega$ be the boundary of the domain, and let Γ_D and Γ_N be the Dirichlet and Neumann subsets. We use notation $\mathcal{A}_\Gamma$ (or a_Γ) to denote the subset of $\mathcal{A}$ (or a) that intersect Γ .

We use symbol $|a|$ to denote the size of a geometrical element (i.e. $|k|$ is the volume of cell k , $|f|$ is the area of face f and $|e|$ is the length of edge e). The same symbol denotes cardinality when applied to a given set. Finally, we introduce operator $\text{dof}(a)$ which maps entity a to its corresponding degree-of-freedom indices. When applied to a set, the operator reads

$$\text{dof}(\mathcal{A}) = \bigcup_{a \in \mathcal{A}} \text{dof}(a)$$

We are interested in the cell-wise contribution of (3)

$$\int_k \nabla \cdot \nabla u \, dk = \int_k g \, dk \quad k \in \mathcal{K} \quad (4)$$

3.1 Finite volume method

We focus on cell-centered, two-point flux approximation, finite volume scheme, for which we approximate p as

$$p \approx \sum_{k \in \mathcal{K}} p_k \xi_k(\mathbf{x}),$$

where p_k is a piecewise-constant value on the cell and

$$\xi_k(\mathbf{x}) = \begin{cases} 1 & \mathbf{x} \in k \\ 0 & \mathbf{x} \notin k \end{cases}$$

We introduce the flux $\mathbf{v} = -\nabla p$. Focusing on the left-hand-side of equation (4), we apply the divergence theorem:

$$\oint_{k_\Gamma} \mathbf{v} \cdot \mathbf{n} \, dk_\Gamma - \int_k g \, dk = 0 \quad k \in \mathcal{K}, \quad (5)$$

and approximate the boundary integral as

$$\oint_{k_\Gamma} \mathbf{v} \cdot \mathbf{n} \, d\Gamma = \sum_{f \in \mathcal{F}_k} v_{kf}.$$

where the face flux v_{kf} is computed with the midpoint rule as

$$v_{kf} = \int_f \mathbf{v} \cdot \mathbf{n} \, df = |f| \nabla p_f \cdot \mathbf{n}_f$$

where $p_f = p(\mathbf{c}_f)$, with $\mathbf{c}_f$ being the centroid of face f and $\mathbf{n}_f$ its normal. The gradient of the potential is computed as

$$v_{kf} \approx |f| \frac{\mathbf{d}_{kf}}{\|\mathbf{d}_{kf}\|^2} \cdot \mathbf{n}_f (p_k - p_f) = T_{kf} (p_k - p_f)$$

where

$$T_{kf} = |f| \frac{\mathbf{d}_{kf}}{\|\mathbf{d}_{kf}\|^2} \cdot \mathbf{n}_f$$

is the half-face transmissibility, i.e., transmissibility of face f on the side of element k . $\mathbf{d}_{kf}$ is the vector connecting the centroid of cell k with the centroid of face f . If k' is the neighboring element of k , i.e. sharing face f , for continuity reasons it follows that

$$v_{kf} = v_{k'f} \quad \rightarrow \quad T_{kf}(p_k - p_f) = T_{k'f}(p_{k'} - p_f)$$

We can eliminate the interface pressure and recover the flux between two neighboring cells as

$$v_{kk'} = T_f(p_k - p_{k'})$$

with the *full* face transmissibility T_f defined as

$$T_f = \left(T_{kf}^{-1} + T_{k'f}^{-1}\right)^{-1}$$

We go back to equation (5) and distinguish internal and boundary flux contributions:

$$\sum_{k' \in \mathcal{K}^k} v_{kk'} + \sum_{f \in \mathcal{F}_f^k} v_{kf} = g_k |k| \quad k \in \mathcal{K} \quad (6)$$

here, $\mathcal{K}^k$ is the set of neighbors of k , while $\mathcal{F}_f^k$ is the set of faces of k lying on the boundary. Neumann boundary fluxes replace v_{kf} , while Dirichlet boundary value replace u_f in v_{kf} . External source terms can be prescribed setting the value of g_k .

The resulting linear system is solved for each cell-centered value of p . Using a MATLAB-style notation, the entries for matrix A read

$$A(k, :) = \begin{cases} A(k, k) = \sum_f T_f & f \in \mathcal{F}^k \\ A(k, k') = -T_f & f \in \mathcal{F}^k \cap \mathcal{F}^{k'} \end{cases} \quad (7)$$

For a FV-TPFA discretization made of hexahedral cells (which is the only supported shape for FV-TPFA, at the moment), A is characterized by a seven-point stencil. There is also an additional contribution to the diagonal entry for cells lying on the Dirichlet boundary

$$A(k, k') = A(k, k') + T_{kf} \quad \text{with } f \in \mathcal{F}_{\Gamma_D}^k \quad (8)$$

Each component of the vector corresponding to the source term reads $\mathbf{f}_k = f_k |k|$.

3.2 Finite elements

For finite elements, we approximate the solution u of (1) as

$$u \approx \sum_{i \in \mathcal{N}} u_i \xi_i(\mathbf{x}) \quad (9)$$

where the nodal basis function ξ_i is such that it is equal to one at node i and zero at all other nodes.

$$\xi_i(\mathbf{x}) = \delta_{ij}$$

Moreover, ξ_i has local support, i.e.

$$\xi_i|_k = 0 \quad \text{if } k \notin \mathcal{K}^i$$

According to classical FEM theory, we rewrite (4) as

$$\int_{\Omega} \xi_i \nabla \cdot \nabla u \, d\Omega = \int_{\Omega} \xi_i g \, d\Omega \quad k \in \mathcal{K}, \quad i \in \mathcal{N}^k$$

Using definition (9) and integration by parts we obtain

$$\sum_{j \in \mathcal{N}} \left(\int_{\Omega} \nabla \xi_i \nabla \xi_j d\Omega \right) u_j = \int_{\Omega} \xi_i g d\Omega \quad i \in \mathcal{N} \quad (10)$$

Equation (10) reveals how to compute the linear system:

$$A_{ij} = \sum_{k \in \mathcal{K}} A_{ij}|_k = \sum_{k \in \mathcal{K}} \int_k \nabla \xi_i|_k \nabla \xi_j|_k dk \quad i, j \in \mathcal{N}^k \quad (11)$$

$$g_i = \sum_{k \in \mathcal{K}} g_i|_k = \sum_{k \in \mathcal{K}} \int_k \xi_i|_k g dk \quad i \in \mathcal{N}^k \quad (12)$$

In the equations above, we indicated with $(\bullet)|_k$ a quantity restricted to a cell. In the following, we will drop this notation whenever it is clear that the quantities are local to a cell.

In case of multi-components dof, we will interchange two different ways of writing the FEM linear combination, using either scalar-valued or vector-valued basis functions:

$$\mathbf{u} \approx \sum_{i \in \mathcal{N}} \mathbf{u}_i \xi_i(\mathbf{x}) = \sum_{d \in \text{dof}(\mathcal{N})} u_d \boldsymbol{\xi}_d(\mathbf{x}) \quad (13)$$

3.2.1 Implementation details

GReS currently implements 4-node tetrahedra, 8-node hexahedra and 27-node hexahedra. In this section, we describe how to compute the integrals appearing in the local contributions (11) and (12) for these finite element shapes.

Numerical integration is performed with Gauss quadrature, as usual. Let n_G be the number of Gauss points in the element. Recall that $|\mathcal{N}^k|$ denotes the number of nodes of each element according to our notation. We introduce the matrix N , of size $n_G \times |\mathcal{N}^k|$, collecting the value of nodal basis functions at each gauss point:

$$N(g, i) = \xi_i(\mathbf{x}_g) \quad (14)$$

The gradient of the local basis functions is collected in a 3-dimensional matrix, ∇N , of size $3 \times |\mathcal{N}^k| \times n_G$, with values stored as:

$$\nabla N(1, i, g) = \frac{\partial \xi_i}{\partial x}(\mathbf{x}_g), \quad (15)$$

$$\nabla N(2, i, g) = \frac{\partial \xi_i}{\partial y}(\mathbf{x}_g), \quad (16)$$

$$\nabla N(3, i, g) = \frac{\partial \xi_i}{\partial z}(\mathbf{x}_g) \quad (17)$$

Tetrahedra The basis function $\xi_i(x, y, z)$, serving as the approximant for the tetrahedral element, is defined as:

$$\xi_i(x, y, z) = a_i + b_i x + c_i y + d_i z \quad i = 1, \dots, 4$$

where the coefficient in this equation, see fig. 1, is given by

$$\begin{bmatrix} a_1 & a_2 & a_3 & a_4 \\ b_1 & b_2 & b_3 & b_4 \\ c_1 & c_2 & c_3 & c_4 \\ d_1 & d_2 & d_3 & d_4 \end{bmatrix} = \begin{bmatrix} 1 & x_1 & y_1 & z_1 \\ 1 & x_2 & y_2 & z_2 \\ 1 & x_3 & y_3 & z_3 \\ 1 & x_4 & y_4 & z_4 \end{bmatrix}^{-1}$$

The volume of any tetrahedron is given by:

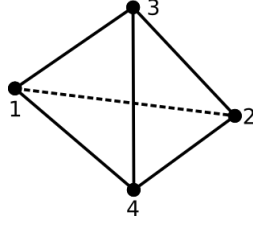


Figure 1: Node distribution for the tetrahedral element.

$$|k| = \frac{1}{6} \det \begin{bmatrix} 1 & x_1 & y_1 & z_1 \\ 1 & x_2 & y_2 & z_2 \\ 1 & x_3 & y_3 & z_3 \\ 1 & x_4 & y_4 & z_4 \end{bmatrix}$$

We immediately observe that

$$\nabla N(:, :, g) = \begin{bmatrix} b_1 & b_2 & b_3 & b_4 \\ c_1 & c_2 & c_3 & c_4 \\ d_1 & d_2 & d_3 & d_4 \end{bmatrix}$$

The gradient of linear basis functions is constant in the tetrahedra, hence the integral in (11) can be computed exactly with $n_G = 1$.

Hexahedra We discuss here only the finite element implementation of trilinear 8-node hexahedra. The extension to triquadratic 27-node hexahedra is straightforward.

We define a reference hexahedron $\hat{k}$ with nodal coordinates $\hat{x}_i, \hat{y}_i, \hat{z}_i = \pm 1$ for $i = 1, \dots, 8$. The basis functions $\xi_i(\hat{x}, \hat{y}, \hat{z})$ in the local hexahedron read:

$$\xi_i(\hat{x}, \hat{y}, \hat{z}) = \frac{1}{8} (1 + \hat{x}\hat{x}_i)(1 + \hat{y}\hat{y}_i)(1 + \hat{z}\hat{z}_i) \quad i = 1, \dots, 8 \quad (18)$$

We denote by $\hat{N}$ and $\nabla \hat{N}$ the counterpart of the basis function matrices in the reference space. These matrices are computed once when instantiating a `FiniteElementType` class object, and assigned to the properties `Nref` and `Jref`, respectively.

To compute matrices N and ∇N in the real (or physical) hexahedra k , we need to use an isoparametric map Ψ that performs a change of coordinates to transform a point in the reference element $\hat{\mathbf{x}} \in \hat{k}$ to a point in the real element $\mathbf{x} \in k$. In symbols:

$$\begin{aligned} \Psi : \mathbb{R}^3 &\longrightarrow \mathbb{R}^3 \\ \hat{\mathbf{x}} \in \hat{k} &\longmapsto \mathbf{x} = \Psi(\hat{\mathbf{x}}) \in k. \end{aligned}$$

For a given Gauss point reference coordinate $\mathbf{x}_g$, we can express the isoparametric transformation Ψ with the aid of the FEM basis functions as

$$\mathbf{x}_g = \Psi(\hat{\mathbf{x}}_g) = \begin{bmatrix} x_1 & x_2 & \dots & x_8 \\ y_1 & y_2 & \dots & y_8 \\ z_1 & z_2 & \dots & z_8 \end{bmatrix} \hat{N}(:, :, g) = X \hat{N}(:, :, g)$$

where X stores the real coordinates of the cell nodes. We can express the gauss point value of the derivatives of the element basis functions in the physical space in terms of those defined in the reference space as follows:

$$\nabla N(:, :, g) = J_\Psi^{-1}(:, :, g) \nabla \hat{N}(:, :, g) \quad g = 1, \dots, n_G \quad (19)$$

Here, J_Ψ is the Jacobian of the isoparametric transformation, computed as

$$J_\Psi(:, :, g) = \nabla \hat{N}(:, :, g) X$$

Using Gauss integration on the reference element, we can compute the local element contribution $A|_k$ to the global linear system matrix A as follows

$$A|_k = \sum_{g=1}^{n_G} w_g \nabla N^T(:, :, g) \nabla N(:, :, g) \det J_\Psi(:, :, g) \quad (20)$$

where w_g is the Gauss point weight. Recall that

$$|k| = \sum_{g=1}^{n_G} w_g \det J_\Psi(:, :, g).$$

Local FEM computations as in (20) are performed in GReS with the calls:

```
>> Atmp = pagetimes(gradN, 'ctranspose', gradN, 'none') ;
```

followed by a summation along the third dimension using

```
>> Aloc = sum(Atmp.*dJWeighed,3);
```

The 3D array `dJWeighed` stores the weighted Jacobian determinants.

3.3 Time integration

From the discrete problem (2), we consider a semi-discrete time dependent variation

$$A\mathbf{u} + M \frac{\partial \mathbf{u}}{\partial t} = \mathbf{g}$$

GReS discretizes equations in time using the implicit backward Euler implicit scheme. Given a time interval $[t, t + \Delta t]$, we approximate the time derivative at time $t + \Delta t$ as

$$\frac{\partial \mathbf{u}}{\partial t} = \frac{\mathbf{u}_{t+\Delta t} - \mathbf{u}_t}{\Delta t},$$

obtaining

$$\left(A + \frac{M}{\Delta t} \right) \mathbf{u}_{t+\Delta t} = \frac{M}{\Delta t} \mathbf{u}_t + \mathbf{g}_{t+\Delta t}.$$

We will later adopt the time difference operator $\Delta_t(\bullet) = (\bullet)_{t+\Delta t} - (\bullet)_t$.

3.4 Nonlinear solution

The system of equations originating from the problem at hand can be nonlinear. GReS uses a classical Newton iterative scheme to address the non-linearities.

Let $\mathbf{R}(u_{t+\Delta t}) = \mathbf{0}$ be the system of nonlinear equations discretized in space and time. At each time step, the m -th Newton iteration reads

$$J^m \delta \mathbf{u} = -\mathbf{R}(u_{t+\Delta t}^{(m)}) \quad (21)$$

with

$$\mathbf{u}_{t+\Delta t}^{(m+1)} = \mathbf{u}_{t+\Delta t}^{(m)} + \delta \mathbf{u}$$

and the Jacobian matrix

$$J^m = \frac{\partial \mathbf{R}(u_{t+\Delta t})}{\partial \mathbf{u}_{t+\Delta t}}$$

In GReS, if a model is linear, we still assemble system (21) to perform a Newton loop, and expect convergence in one single iteration.

4 Physics models

Section 3 builds on the Poisson problem to introduce the discretization techniques implemented in GReS in the simplest setting.

In this section, we apply those concepts to specific physical models. For each model, we will:

1. present the governing differential equations. It will be implicitly assumed that the problem is closed by appropriate boundary conditions and initial conditions;
2. describe the physical meaning of the variables and material parameters involved. The base physical dimensions adopted in the dimensional analysis are: $L \rightarrow$ length, $P \rightarrow$ pressure, $T \rightarrow$ time;
3. introduce the constitutive relationships;
4. derive the fully discretized system of equations, extending the content of section 3 when necessary;
5. discuss solver-specific solution strategies, if any.
6. All variables will be defined locally, within each sub-section. This means that a symbol can have different meanings in different physics.

Each subsection is named after the corresponding GReS `PhysicsSolver` class being described.

4.1 Poisson

Poisson problem

$$\nabla \cdot \nabla u = g \quad \text{in } \Omega$$

has already been covered in the previous section. In GReS, the user can prescribe a manufactured solution for a given forcing term $g(x)$. With this information, the user can compute the L^2 and H^1 error norm of the problem, which is useful for convergence studies.

4.2 SinglePhaseFlow

This model discretizes single-phase slightly compressible flow in porous media. The governing equations are

$$S \frac{\partial p}{\partial t} + \nabla \cdot v = q \tag{22}$$

where:

- p is the absolute pressure [P].
- $S = \alpha + \phi\beta$ is the storage coefficient [1/P];
- ϕ is porosity [/];
- $\alpha = d\phi/dp$ is the rock compressibility [1/P];
- $\beta = (1/\rho)(d\rho/dp)$ is the fluid compressibility [1/P] with ρ being its density;
- q is the source/sink, i.e., the fluid flow injected or produced per unit volume [1/T].
- γ is the specific weight of the fluid [P/L]

Darcy's velocity, v , represents the superficial fluid velocity through a porous medium, with

$$v = -\frac{\mathbf{K}}{\mu} \nabla (p + \gamma z)$$

where:

- $\mathbf{K}$ is the second-order intrinsic permeability tensor $[\text{L}^2]$;
- μ is the dynamic viscosity $[\text{P} \times \text{T}]$.

Note that the sign of the elevation head term is consistent with the z axis positive upward. The flow equation:

$$\nabla \cdot \left[\frac{\mathbf{K}}{\mu} \nabla (p + \gamma z) \right] = S \frac{\partial p}{\partial t} - q$$

is *linear*.

At time $t + \Delta t$, the resulting discretized system of equations is

$$\left(H + \frac{P}{\Delta t} \right) \mathbf{p}_{t+\Delta t} = \frac{P}{\Delta t} \mathbf{p}_t + \mathbf{q}$$

Matrix H is referred to as *stiffness matrix*, while P is the *capacity matrix*. As already stated in section 3.4, despite the linearity of the model, we still solve for a Newton increment $J\delta\mathbf{p} = -\mathbf{R}$, with

$$J = H + \frac{P}{\Delta t}$$

and expect convergence in one iteration.

Single phase flow can be discretized with both FV-TPFA and FEM.

SinglePhaseFlowFVTPFA The half-face transmissibility reads

$$T_{kf} \approx |f| \frac{K_k}{\mu_k} \frac{\mathbf{d}_{kf}}{\|\mathbf{d}_{kf}\|_2^2} \cdot \mathbf{n}_f$$

H can then be assembled as A in equation (7). P is diagonal with entries:

$$P(k, k) = S_k |k| \quad \forall k \in \mathcal{K} \quad (23)$$

Each entry of the residual $\mathbf{R}$ reads

$$\mathbf{R}(k) = S_k |k| \frac{\Delta_t(p_k)}{\Delta t} + \sum_{k' \in \mathcal{K}^k} (v_{kk'}^p + v_{kk'}^z) + \sum_{f \in \mathcal{F}_1^k} (v_{kf}^p + v_{kf}^z) - q_k |k| \quad k \in \mathcal{K}$$

where

$$v_{kk'}^p = T_f(p_k - p_{k'}) \quad v_{kk'}^z = T_f \gamma (z_k - z_{k'}) \quad (24)$$

$$v_{kf}^p = T_{kf}(p_k - p_f) \quad v_{kf}^z = T_{kf} \gamma (z_k - z_f) \quad (25)$$

SinglePhaseFlowFEM

$$H_{ij}|_k = \frac{1}{\mu_k} \int_k \nabla \xi_i K_k \nabla \xi_j dk \quad i, j \in \mathcal{N}^k$$

$$P_{ij}|_k = S_k \int_k \xi_i \xi_j dk \quad i, j \in \mathcal{N}^k$$

4.3 VariablySaturatedFlow

The mathematical formulation for variably saturated, slightly compressible flow in porous media is given as

$$S(s) \frac{\partial p}{\partial t} + \nabla \cdot \mathbf{v} = q \quad (26)$$

where the storage coefficient, $S(s)$, is a function of the water saturation s (where $0 \leq s \leq 1$) and is defined as:

$$S(s) = s(\alpha + \phi\beta) + \phi \frac{ds}{dp}$$

Darcy's velocity is modified to account for the simultaneous presence of air and water in the pore space:

$$v = -\lambda \mathbf{K} \nabla (p + \gamma z) \quad (27)$$

where $\lambda = \kappa_r(s)/\mu$ is the so-called mobility factor and $\kappa_r(s)$ is the relative permeability. Substituting eq. (27) into (26) gives the classical pressure form of the Richards' equation:

$$\nabla \cdot [\lambda(s) \mathbf{K} \nabla (p + \gamma z)] = S(s) \frac{\partial p}{\partial t} - q. \quad (28)$$

Denoting s_e as the effective water saturation, given by:

$$s_e = \frac{s - s_r}{s_s - s_r}$$

where, s_r denotes the residual water saturation, and s_s represents the maximum water saturation, typically taken as 1.

The relationships by capillary pressure-saturation define the link between pressure and effective saturation, generally in a nonlinear form, and are expressed as:

$$s_e = s_e(s) \quad (29)$$

and in a similar way, another nonlinear relationship is given by the relative permeability and the pressure:

$$\kappa_r = \kappa_r(s) \quad (30)$$

In GReS, the relationships in (29) and (30) can be provided in two different ways: through tables or analytical curves. For the table-based approach, these properties are determined by performing linear interpolation on the values provided in the tables. For the analytical approach, the relationship between capillary pressure and water saturation is

$$s_e = (1 + (\beta p)^n)^{-m}$$

where β is a parameter related to the average size of pores, the exponents n and m are related to the pore-size distribution. This model was proposed by van Genuchten [2–4]. As for the relative permeability, it can be described using the van Genuchten–Mualem model, expressed as:

$$\kappa_r = s_e^k \left[1 - \left(1 - s_e^{1/m} \right)^m \right]^2, \quad m = 1 - 1/n$$

or the model van Genuchten-Burdine:

$$\kappa_r = s_e^2 \left[1 - \left(1 - s_e^{1/m} \right)^m \right], \quad m = 1 - 2/n$$

and k is a connectivity factor, usually 0.5.

Variably saturated flow can only be discretized with FV-TPFA.

4.4 Poromechanics

The **Poromechanics** module solves the balance of linear momentum, in the assumption of small strain

$$\nabla \cdot \sigma = -F$$

where:

- σ is the stress tensor [P];
- F is a volume force [P/L];

We introduce the differential constitutive relationship

$$d\sigma = D(\sigma) : d\varepsilon$$

where:

- $\varepsilon = \nabla^s \mathbf{u}$ is the strain tensor [I]
- $\mathbf{u}$ is the displacement vector [L]
- $D(\sigma)$ is the material tangent operator, a 4th-order tensor [P].

For small strains and linear elastic materials, $D = \text{const}$ and the constitutive relationship becomes

$$\sigma = D : \varepsilon$$

Using FEM, the discrete variational formulation of the linear momentum balance reads

$$\sum_{j \in \mathcal{N}} \left(\int_{\Omega} \nabla^s \xi_i D \nabla^s \xi_j d\Omega \right) \mathbf{u}_j = \int_{\Omega} \xi_i \cdot \mathbf{F} d\Omega \quad i \in \mathcal{N} \quad (31)$$

which leads to the final assembled system

$$K \mathbf{u} = \mathbf{F}$$

K is the *mechanical stiffness matrix*. The problem is solved in terms of displacements degrees of freedom (DoF), with $d = 3$ components for each node of the discretization. The problem is implemented using engineering Voigt notation, to express 2-nd order symmetric tensors as 6×1 vectors, and 4-th order tensors as 6×6 matrices. The strain tensor in engineering Voigt notation reads

$$\varepsilon = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, 2\varepsilon_{yz}, 2\varepsilon_{xz}, 2\varepsilon_{xy}]$$

For multi-components DoF, the basis function matrix (14) becomes 3D with size $d \times (d \cdot |\mathcal{N}^k|) \times n_G$:

$$N(:, d(i-1) + 1 : di, g) = \begin{bmatrix} \xi_i(\mathbf{x}_g) & 0 & 0 \\ 0 & \xi_i(\mathbf{x}_g) & 0 \\ 0 & 0 & \xi_i(\mathbf{x}_g) \end{bmatrix}$$

Similarly, we can use a matrix form to express the symmetric gradient of the displacements basis functions in Voigt notation. We define matrix B , of size $6 \times 3 \cdot |\mathcal{N}^k| \times n_G$

$$B(:, d(i-1) + 1 : di, g) = \begin{bmatrix} \frac{\partial \xi_i}{\partial x}(\mathbf{x}_g) & 0 & 0 \\ 0 & \frac{\partial \xi_i}{\partial y}(\mathbf{x}_g) & 0 \\ 0 & 0 & \frac{\partial \xi_i}{\partial z}(\mathbf{x}_g) \\ 0 & \frac{\partial \xi_i}{\partial z}(\mathbf{x}_g) & \frac{\partial \xi_i}{\partial y}(\mathbf{x}_g) \\ \frac{\partial \xi_i}{\partial z}(\mathbf{x}_g) & 0 & \frac{\partial \xi_i}{\partial x}(\mathbf{x}_g) \\ \frac{\partial \xi_i}{\partial y}(\mathbf{x}_g) & \frac{\partial \xi_i}{\partial x}(\mathbf{x}_g) & 0 \end{bmatrix}$$

such that

$$\varepsilon(\mathbf{x}_g) = B(:, :, g) \mathbf{u}_k, \quad \mathbf{u}_k = [\mathbf{u}_1, \dots, \mathbf{u}_{|\mathcal{N}^k|}]^T$$

We can compute the elemental contribution to the stiffness matrix as

$$K|_k = \sum_{g=1}^{n_G} w_g B_g^T D_g(\mathbf{u}) B_g \det J_{\Psi,g}, \quad k \in \mathcal{K}$$

We use a shorter notation $B_g = B(:, :, g)$ to indicate pages of 3D matrix associated with Gauss point g . For application of Newton algorithm, we have

$$J^{(m)} = K(\mathbf{u}_{t+\Delta t}^{(m)})$$

while the residual is

$$\mathbf{R}^{(m)} = \mathbf{f}_{int}^{(m)}(\mathbf{u}) - \mathbf{F}$$

The vector of internal forces is computed locally during the assembly loop as

$$\mathbf{f}_{int}|_k = \sum_{g=1}^{n_G} w_g B_g^T [\sigma(\mathbf{x}_g) - \sigma_0(\mathbf{x}_g)] \det J_{\Psi,g}$$

where σ_0 is any given initial stress. At each Newton solve m , the value of D^{ep} and $\sigma(\varepsilon)$ is obtained from constitutive update algorithms, implemented in the `updateMaterial` function of each Material class. GReS updates the stress using the strain $\varepsilon^{(m)}$ cumulated at the current time step, that is

$$\varepsilon^m = \nabla^s(\mathbf{u}_{t+\Delta t}^{(m)} - \mathbf{u}_t)$$

4.5 EmbeddedFractureMechanics

This physical model allows to introduce embedded fractures in the 3D discretization. Each fracture is equipped with a frictional contact mechanics law. This module implements a non-Galerkin formulation with piecewise constant enrichment for the displacement discontinuities [1].

Let Γ_f be the internal fracture and let $\mathcal{M}$ be the portion of the domain Ω intercepted by the fracture. Cut-cells (i.e. cells being intersected by Γ_f) are denoted as $\tilde{k} \in \tilde{\mathcal{K}}$, while the corresponding intersection is $\tilde{f}$. We model fractures as planar parallelogram surfaces, and we assume each cut element to be fully intersected by the plane. The local enrichment basis functions $\boldsymbol{\eta}|_{\tilde{k}}$ are piecewise constant and coincide with the local component of motion around the fracture. Given the local normal $\mathbf{n}$ and the local tangential components $\mathbf{m}_1, \mathbf{m}_2$, we have

$$\boldsymbol{\eta}_1 = \mathbf{n}, \quad \boldsymbol{\eta}_2 = \mathbf{m}_1, \quad \boldsymbol{\eta}_3 = \mathbf{m}_2 \quad (32)$$

FEM interpolation allows to decompose the enriched displacements field $\bar{\mathbf{u}}$ as

$$\bar{\mathbf{u}} \approx \mathbf{u} + \tilde{\mathbf{u}} = \sum_{i \in \text{dof}(\mathcal{N})} u_i \boldsymbol{\xi}_i + \sum_{b \in \text{dof}(\tilde{\mathcal{K}})} w_b \boldsymbol{\phi}_b \quad (33)$$

Where the introduction of the local discontinuous basis functions $\boldsymbol{\phi}$

$$\boldsymbol{\phi}_b(x)|_{\tilde{k}} = H(x) \boldsymbol{\eta}_b(x) - \sum_{l=1}^{|\mathcal{N}^{\tilde{k}}|} H(\mathbf{x}_l) \xi_l(\mathbf{x}_l) \boldsymbol{\eta}_b(\mathbf{x}_l) \quad \tilde{k} \in \tilde{\mathcal{K}} \quad (34)$$

allows to isolate a continuous part of the displacement field which is interpolated with classical lagrangian basis functions. $H(\bullet)$ is the Heaviside function, whose value is either 0 or 1 depending on the position of each nodes w.r.t the internal fracture,

The weak form of the problem reads:

$$\int_{\Omega} \nabla^s \boldsymbol{\xi}_i : \sigma d\Omega - \int_{\Omega} \boldsymbol{\xi}_i \cdot \mathbf{F} d\Omega = 0 \quad i \in \mathcal{N} \quad (35)$$

$$\int_{\Gamma_f} (\beta \otimes \mathbf{n}_l)^s : \sigma d\Gamma_f - \int_{\Gamma_f} \boldsymbol{\eta}_l \cdot \mathbf{t} d\Gamma_f = 0 \quad l \in \mathcal{K}_{\Gamma_f} \quad (36)$$

For details see [1]. The first equation is the classical linear momentum balance. The second equation represents the equilibrium on the fracture element. Operator $\beta_b \otimes \mathbf{n}$ maps a stress computed in the bulk of the cut-cell to traction $\mathbf{t}$ in the fracture. The following constitutive relations holds for the stress tensor:

$$\sigma = D : \nabla^s \bar{\mathbf{u}} = D : (\nabla^s \mathbf{u} + \nabla^s \tilde{\mathbf{u}}). \quad (37)$$

The total strain can be expressed as

$$\nabla^s \bar{\mathbf{u}} = \sum_{i \in \text{dof}(\mathcal{N})} u_i \nabla^s \boldsymbol{\xi}_i + \sum_{b \in \text{dof}(\tilde{\mathcal{K}})} w_b \nabla^s \phi_b \quad (38)$$

Matrix $\tilde{B}$ is the enhanced strain operator, which mimics the continuous strain operator

$$\nabla^s \tilde{\mathbf{u}}(\mathbf{x}_g) = \tilde{B}(:, :, g) \mathbf{w}_{\tilde{k}}, \quad \tilde{k} \in \tilde{\mathcal{K}}, \quad g = 1, \dots, n_G$$

It is a $6 \times 3 \times n_G$ matrix defined in Voigt notation as

$$\tilde{B} = \begin{bmatrix} \mathbf{n} \otimes_s \mathbf{v} \\ \mathbf{m}_1 \otimes_s \mathbf{v} \\ \mathbf{m}_2 \otimes_s \mathbf{v} \end{bmatrix}^T \quad (39)$$

where

$$\mathbf{v} = \sum_{i=1}^{|\mathcal{N}^{\tilde{k}}|} H(\mathbf{x}_i) \nabla N(:, \text{dof}(i), :) \quad (40)$$

Operator $\beta_b \otimes \mathbf{n}$ is expressed in matrix form as:

$$E = \frac{|\tilde{f}|}{|\tilde{k}|} \begin{bmatrix} \mathbf{n} \otimes_s \mathbf{n} \\ \mathbf{m}_1 \otimes_s \mathbf{n} \\ \mathbf{m}_2 \otimes_s \mathbf{n} \end{bmatrix}^T \quad (41)$$

of size 3×6 . The scaling factor $|\tilde{f}|/|\tilde{k}|$ restores consistency between the integrals and guarantees exact solution for uniform stress states (patch test).

The linearized system at Newton iteration m reads

$$\mathbf{J}^{(m)} = \begin{bmatrix} A_{uu} & A_{uw} \\ A_{wu} & A_{ww} \end{bmatrix}$$

where $A_{uu} = K$ from the **Poromechanics** model and

$$A_{uw}|_{\tilde{k}} = \sum_{g=1}^{n_G} w_g B_g^T D_g(\mathbf{u}) \tilde{B}_g \det J_{\Psi,g}, \quad \tilde{k} \in \tilde{\mathcal{K}} \quad (42)$$

$$A_{wu}|_{\tilde{k}} = \sum_{g=1}^{n_G} w_g E D_g(\mathbf{u}) B_g \det J_{\Psi,g}, \quad \tilde{k} \in \tilde{\mathcal{K}} \quad (43)$$

$$A_{ww}|_{\tilde{k}} = \sum_{g=1}^{n_G} w_g E D_g(\mathbf{u}) \tilde{B}_g \det J_{\Psi,g} - |\tilde{f}| T, \quad \tilde{k} \in \tilde{\mathcal{K}} \quad (44)$$

We will provide definition of matrix T in the next section. The internal forces for the residual vector are assembled as in **Poromechanics**, using the definition of σ in (37).

4.5.1 Contact mechanics

The enrichment in the embedded fracture is equipped with frictional contact mechanics law to allow for opening and sliding of the fracture. The opening mechanics is governed by the law

$$t_N = \mathbf{t} \cdot \mathbf{n}_f \leq 0, \quad g_N = \llbracket \mathbf{u} \rrbracket \cdot \mathbf{n}_f \geq 0, \quad t_N g_N = 0$$

while sliding is governed by the Coulomb friction law

$$\|\mathbf{t}_T\|_2 \leq \tau_{lim}(t_N), \quad \Delta_t(\mathbf{g}_T) \cdot \mathbf{t}_T - \tau_{lim}(t_N) \|\Delta_t(\mathbf{g}_T)\|_2 = 0$$

where $\tau_{lim} = c - t_N \tan \theta$, where c is the cohesion [P] and θ is the friction angle. The tangential traction vector is $\mathbf{t}_T = (I - \mathbf{n}_f \otimes \mathbf{n}_f) \cdot \mathbf{t}$. We distinguish the contact state of each fracture element as:

- *stick*, when $t_N < 0$ and the tangential contact conditions are satisfied;
- *slip*, when $t_N < 0$ but the tangential traction violates the Coulomb criterion, reaching the limiting value

$$\mathbf{t}_T^* = \tau_{lim}(t_N) \frac{\Delta_t(\mathbf{g}_T)}{\|\Delta_t(\mathbf{g}_T)\|_2} \quad (45)$$

and allowing for relative slip between the two sides of the fracture element;

- *open*, when $t_N > 0$ and no contact takes place.

To keep the formulation purely displacement based, we need a relationship between traction and displacement. We adopt a simple penalty approach and express the traction as:

$$t_N = \epsilon_N g_N$$

$$\begin{cases} \mathbf{t}_T = \epsilon_T \Delta_t(\mathbf{g}_T) & \text{if } \|\mathbf{t}_T\| \leq c - \tan(\theta) \epsilon_N g_N \\ \mathbf{t}_T = (c - \tan(\theta) \epsilon_N g_N) \frac{\Delta_t(\mathbf{g}_T)}{\|\Delta_t(\mathbf{g}_T)\|} & \text{if } \|\mathbf{t}_T\| \geq c - \tan(\theta) \epsilon_N g_N \end{cases}$$

Note that, given the definition of the local enrichment, we have for each fracture element:

$$g_N = w_1, \quad \|\Delta_t(\mathbf{g}_T)\| = \sqrt{w_2^2 + w_3^2} \quad (46)$$

Finally, we can express matrix T , which changes according to the state of the fracture element

$$T|_{\tilde{k}} = \frac{\partial t}{\partial \mathbf{g}} \Big|_{\tilde{k}} = \begin{cases} \text{Open:} & \mathbf{0} \\ \text{Stick:} & \begin{bmatrix} \epsilon_N & 0 & 0 \\ 0 & \epsilon_T & 0 \\ 0 & 0 & \epsilon_T \end{bmatrix} \\ \text{Slip:} & \begin{bmatrix} \epsilon_N & 0 & 0 \\ -\epsilon_N \tan \theta \frac{g_{T,1}}{\|\mathbf{g}_T\|} & \tau_{max} \frac{g_{T,2}^2}{\|\mathbf{g}_T\|^3} & -\tau_{max} \frac{g_{T,1} g_{T,2}}{\|\mathbf{g}_T\|^3} \\ -\epsilon_N \tan \theta \frac{g_{T,2}}{\|\mathbf{g}_T\|} & -\tau_{max} \frac{g_{T,2} g_{T,1}}{\|\mathbf{g}_T\|^3} & \tau_{max} \frac{g_{T,1}^2}{\|\mathbf{g}_T\|^3} \end{bmatrix} \end{cases} \quad (47)$$

The contact algorithm is a classical active-set strategy, where the solution of the problem is obtained with three nested loops:

- a time loop;
- a configuration loop, where at each iteration the contact state is updated. The loop stops when the configuration remains unchanged (up to a prescribed tolerance);
- the newton loop, which solves the nonlinear problem for a fixed configuration;

4.6 Biot coupled poromechanics

4.6.1 Fixed stress split algorithm

5 Interface models

5.1 Mortar method for non-conforming grids

5.2 MeshTying

5.3 SolidMechanicsContact

References

- [1] Matteo Cusini et al. “Simulation of coupled multiphase flow and geomechanics in porous media with embedded discrete fractures”. In: *International Journal for Numerical and Analytical Methods in Geomechanics* 45.5 (2021), pp. 563–584.
- [2] M. Th. van Genuchten. “A Closed-form Equation for Predicting the Hydraulic Conductivity of Unsaturated Soils”. In: *Soil Science Society of America Journal* 44.5 (1980), pp. 892–898. DOI: <https://doi.org/10.2136/sssaj1980.03615995004400050002x>.
- [3] Knut-Andreas Lie. *An Introduction to Reservoir Simulation Using MATLAB/GNU Octave: User Guide for the MATLAB Reservoir Simulation Toolbox (MRST)*. Cambridge University Press, 2019.
- [4] Martinus Van Genuchten and D.R. Nielsen. “On Describing and Predicting the Hydraulic Properties of Unsaturated Soils”. In: *Annales Geophysicae* 3 (Jan. 1985), pp. 615–628.